

GitHub Trending Repository Scraper and Cleaner

Purpose

This module collects daily trending repositories from GitHub's public Trending page (<https://github.com/trending>) and prepares them for downstream real-time analysis.

The result is a cleaned dataset of repository metadata, suitable for Kafka-based streaming and short-term trend visualizations.

Key Features

- Scrapes daily trending repositories directly from the GitHub Trending HTML page
 - Extracts repository name, description, language, star count, and URL
 - Stores historical data as daily snapshots
 - Cleans and normalizes programming language entries
 - Removes duplicates and invalid entries
 - Applies whitelist-based filtering for consistent downstream analysis
-

Output Structure

All data is stored in the `./ScrapedData/` directory.

- `ScrapedData_<DD.MM.YY>.json` : Raw scraped data per day
 - `ScrapData_All_Cleaned.json` : Final cleaned and combined dataset
 - `scraped_languages_unique.csv` : List of unique languages found
 - `scraped_blacklist.csv` : Languages excluded from the final dataset
-

Processing Steps

1. HTML Scraping

- Sends an HTTP request to GitHub Trending (daily view)
- Parses the HTML using `BeautifulSoup`
- Extracts repository metadata from each article block

2. Raw Data Storage

- Each scrape is stored as a separate `.json` file named by date

3. Data Consolidation

- Combines all historical scraped files into a single DataFrame
- Removes duplicates by repository name

4. Cleaning and Normalization

- Removes rows with missing or invalid values (e.g. "N/A", empty strings)
- Parses and validates `scraped_at` timestamps
- Normalizes programming language names using a predefined mapping

5. Language Filtering

- Uses a strict whitelist to filter valid general-purpose programming languages
- All other languages are excluded and logged to `scraped_blacklist.csv`

Inclusion Criteria for Languages

A language is retained if it:

- Supports control structures (e.g., `if`, `while`, `function`)
- Is designed for general-purpose programming
- Is Turing complete or close in expressive power

Examples:

- Included: Python, JavaScript, Java, Rust, Go
- Excluded: HTML, CSS, Dockerfile, JSON

Output Summary

- Cleaned daily trending repositories
- Unified JSON file for streaming simulation
- Language normalization and blacklist tracking
- Logging of total entries before and after cleaning

Notes

- This script is designed for repeated daily execution
- Scraping may break if GitHub changes the structure of the Trending page
- Star counts and languages are scraped from HTML and may be incomplete for some entries

Author

Big Data Engineering
FH Technikum Wien, Summer Semester 2025

Manpreet Misson, Timothy Gregorian, Omar Sidi Mammar

Imports and Configuration

Load required libraries and define output folder & current date.

```
In [1]: import os
import json
import requests
import pandas as pd
from bs4 import BeautifulSoup
from datetime import datetime

scraped_dir = "./ScrapedData"
today_str = datetime.now().strftime("%d.%m.%y")
```

Scraping GitHub Trending Page

Scrapes the public GitHub Trending page for currently popular repositories. Extracts repository name, description, language, stars, and timestamp.

```
In [2]: url = "https://github.com/trending?since=daily"
print(f"Scraping: {url}")
response = requests.get(url)
if response.status_code != 200:
    print("Error retrieving the page!")
    exit()

soup = BeautifulSoup(response.text, 'html.parser')
repositories = soup.find_all('article', class_='Box-row')

all_repo_data = []
for repo in repositories:
    full_name = repo.h2.a.get('href').strip('/')
    description_tag = repo.p
    description = description_tag.text.strip() if description_tag else "No description"

    lang_tag = repo.find('span', itemprop='programmingLanguage')
    language = lang_tag.text.strip() if lang_tag else "N/A"

    stars_tag = repo.find('a', href=lambda x: x and x.endswith('/stargazers'))
    stars = stars_tag.text.strip().replace(',', '') if stars_tag else "0"

    repo_url = f"https://github.com/{full_name}"
    repo_data = {
        "name": full_name,
        "description": description,
        "language": language,
        "stars": stars,
        "url": repo_url,
        "scraped_at": today_str
    }
    all_repo_data.append(repo_data)
```

Scraping: <https://github.com/trending?since=daily>

Save Raw JSON Data

Saves the scraped GitHub trending repositories as raw JSON for archival.

```
In [3]: raw_data_path = os.path.join(scraped_dir, f"ScrapedData_{today_str}.json")
with open(raw_data_path, "w", encoding="utf-8") as f:
    json.dump(all_repo_data, f, ensure_ascii=False, indent=2)
print(f"Saved raw data to: {raw_data_path}")
```

Saved raw data to: ./ScrapedData/ScrapedData_19.06.25.json

Load All Raw Scraped Files

Loads all previously scraped JSON files into a unified DataFrame.

```
In [4]: scraped_files = [f for f in os.listdir(scraped_dir) if f.endswith(".json")]
all_data = []
for file in scraped_files:
    file_path = os.path.join(scraped_dir, file)
    try:
        with open(file_path, "r", encoding="utf-8") as f:
            data = json.load(f)
            all_data.extend(data)
    except Exception as e:
        print(f"Error loading {file}: {e}")

df = pd.DataFrame(all_data)
print(f"Total loaded entries: {len(df)}")
```

Total loaded entries: 387

Data Cleaning

Handles missing values, trims whitespace, removes duplicates, and parses dates.

```
In [5]: na_values = ["N/A", "n/a", "NaN", "nan", "null", "None", "NONE", "Null", "", " "]
df = df.apply(lambda col: col.str.strip() if col.dtypes == "object" else col)
df.replace(na_values, pd.NA, inplace=True)
df.dropna(subset=["name", "language", "url", "scraped_at"], inplace=True)
df.fillna("", inplace=True)
df = df.drop_duplicates(subset="name")
df["scraped_at"] = pd.to_datetime(df["scraped_at"], format="%d.%m.%y", errors="coerce")
df = df[df["scraped_at"].notna()]
df = df.sort_values(by="scraped_at", ascending=True)
```

Export Unique Languages

Extracts and saves the list of all unique programming languages from the dataset.

```
In [6]: lang_df = pd.DataFrame(sorted(df["language"].unique()), columns=["Language"])
lang_path = os.path.join(scraped_dir, "scraped_languages_unique.csv")
lang_df.to_csv(lang_path, index=False)
print(f"Saved language list to: {lang_path}")
```

Saved language list to: ./ScrapedData/scraped_languages_unique.csv

Define Whitelist of Valid Programming Languages

This helps to isolate programming languages from tools, platforms, or incorrect entries.

```
In [7]: whitelist = {
    "Ada", "APL", "Assembly", "BASIC", "C", "C#", "C++", "COBOL", "Clojure", "Cr
    "D", "Dart", "Delphi", "Elixir", "Elm", "Erlang", "F#", "Fortran", "FreeBASI
    "Groovy", "Hack", "Haskell", "Java", "JavaScript", "Julia", "Kotlin", "Lisp"
    "Matlab", "Nim", "OCaml", "Objective-C", "Objective-C++", "Pascal", "Perl",
    "PowerShell", "Prolog", "Python", "R", "Raku", "Ruby", "Rust", "Scala", "Sch
    "Smalltalk", "Solidity", "Swift", "TypeScript", "VBA", "Visual Basic", "Zig"
}
```

Generate and Save Blacklist

Filters out all languages not on the whitelist and saves them to a file.

```
In [8]: blacklist_df = lang_df[~lang_df["Language"].isin(whitelist)]
blacklist_path = os.path.join(scraped_dir, "scraped_blacklist.csv")
blacklist_df.to_csv(blacklist_path, index=False)
print(f"Saved blacklist to: {blacklist_path}")

blacklist = set(blacklist_df["Language"].str.lower().str.strip())
df = df[~df["language"].str.lower().str.strip().isin(blacklist)].copy()
```

Saved blacklist to: ./ScrapedData/scraped_blacklist.csv

Language Normalization

Maps alternate or inconsistent names to canonical forms (e.g., "Javascript" → "JavaScript").

```
In [9]: normalization_map = {
    "C++11": "C++", "Javascript": "JavaScript", "javascript": "JavaScript",
    "Typescript": "TypeScript", "typescript": "TypeScript", "Golang": "Go",
    "MATLAB": "Matlab", "Matlab": "Matlab", "Perl 6": "Perl",
    "LISP": "Lisp", "Common Lisp": "Lisp", "Lisp": "Lisp", "OCaml": "OCaml",
    "Delphi/Object Pascal": "Delphi", "Visual Basic (.Net)": "Visual Basic",
    "Visual Basic 6": "Visual Basic", "Visual Basic 6.0": "Visual Basic",
    "Visual Basic .NET": "Visual Basic", "VB.NET": "Visual Basic"
}

df["language"] = df["language"].apply(lambda x: normalization_map.get(x.strip(),
```

Save Final Cleaned Dataset

Writes cleaned and normalized repository data back to disk in structured JSON format.

```
In [10]: final_data_path = os.path.join(scraped_dir, "ScrapData_All_Cleaned.json")
df.to_json(final_data_path, orient="records", indent=2, date_format="iso", force
print(f"Saved cleaned data to: {final_data_path}")
print(f"Total remaining entries: {len(df)}")
```

```
Saved cleaned data to: ./ScrapedData/ScrapData_All_Cleaned.json  
Total remaining entries: 138
```